

what difference between middleware and filters interview question

This is a very common ASP.NET Core interview question, especially for experienced developers.

Middleware vs Filters in ASP.NET Core

Middleware	Filters
Works at the HTTP request pipeline level	Works inside the MVC/Web API action pipeline
Runs before/after MVC endpoint execution	Runs around controller/action execution
Applies generally to all requests passing through it	Applies to controllers/actions where configured
Has access to <code>HttpContext</code>	Has access to MVC contexts such as <code>ActionExecutingContext</code>
Good for logging, exception handling, authentication-related pipeline behavior, headers, correlation IDs	Good for authorization, action-specific validation, result modification, action logging
Can short-circuit the entire request pipeline	Can short-circuit MVC/action execution
Created using <code>Use</code> , <code>Run</code> , <code>Map</code> , or custom middleware	Implemented with interfaces/attributes such as <code>IActionFilter</code> , <code>IAsyncActionFilter</code> , <code>IExceptionFilter</code>
Executes before MVC filters are reached	Executes after the request enters the MVC/action pipeline

Simple flow

HTTP Request



```
Middleware 1
  ↓
Middleware 2
  ↓
Routing
  ↓
Controller / MVC Pipeline
  ↓
Authorization Filter
  ↓
Resource Filter
  ↓
Action Filter
  ↓
Controller Action
  ↓
Result Filter
  ↓
HTTP Response
```

Middleware example

Suppose you want to log every incoming API request:

```
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        Console.WriteLine($"Request: {context.Request.Path}");

        await _next(context);

        Console.WriteLine($"Status: {context.Response.StatusCode}");
    }
}
```

Register it:

</> C#



This is appropriate because request logging is a **cross-cutting HTTP concern**.

Filter example

Suppose you want logic specifically before and after controller actions:

```
public class LoggingActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        Console.WriteLine("Before controller action");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("After controller action");
    }
}
```

Use it:

```
[ServiceFilter(typeof(LoggingActionFilter))]
public class EmployeeController : ControllerBase
{
    [HttpGet]
    public IActionResult GetEmployees()
    {
        return Ok();
    }
}
```

Here a filter makes sense because the logic is related specifically to **MVC controller/action execution**.

When should I use which?

Think about the **scope**.

Does the logic concern the entire HTTP request?

|

YES

↓

Middleware

Does the logic specifically concern MVC/controller/action execution?

|



YES
↓
Filter

For example:

Global exception handling → Middleware
Request/response logging → Middleware
Correlation ID → Middleware
Security headers → Middleware

Action-specific validation → Filter
Action logging/auditing → Filter
Result modification → Filter
MVC authorization logic → Filter



One important interview point: **don't say middleware is always global and filters are always local**. Middleware can be conditionally branched, and filters can also be registered globally. The fundamental difference is **where they execute in the ASP.NET Core pipeline**.

Experienced-level interview answer

Middleware works at the HTTP pipeline level, whereas filters work inside the MVC/action execution pipeline. Middleware is suitable for cross-cutting concerns such as global exception handling, request logging, correlation IDs, and security headers. Filters are better when the logic needs MVC-specific context, such as action arguments, controller metadata, action results, or authorization requirements. Both can short-circuit execution, but at different stages of the request pipeline.

Common interview trap

Interviewer: *"If both middleware and exception filters can handle exceptions, which one would you choose?"*

A strong answer:

For application-wide exception handling, I generally prefer **exception-handling middleware** because it can catch exceptions across a broader portion of the HTTP pipeline. An exception filter is more appropriate when exception handling specifically depends on MVC/controller context.

One-line answer to remember:

Middleware = HTTP pipeline. Filter = MVC/Controller pipeline.

ChatGPT can make mistakes. Check important info.

